In Class Exercise: Introducing Concepts of Software Discrimination Testing

Consent Form:

Signup Form:

Software systems deployed in real-world settings may exhibit discriminatory behavior toward certain communities due to biases embedded in their underlying algorithms or datasets. Consequently, software discrimination testing has emerged as an important component of software quality assurance. The goal of this assignment is to examine how newcomers learn software discrimination testing using existing online resources.

We use the NIST ACTS combinatorial testing tool to generate test cases for software discrimination testing

To acquire necessary materials to complete the assignment, download twayDiscriminationTesting.zip

This exercise will be completed using Python. You will need to install the following packages:

- pandas
- numpy
- scikit-learn
- java (brew install openjdk@17 for Mac Users)

You can use any online resource **with the exception of LLM-based conversational agents** to complete this exercise.

Before completing each part in this exercise, make sure to complete the corresponding pre-survey provided later in the description**.** There are also post**-surveys that must be completed after completing each part of the exercise**, and the link is provided later in the description.

Part I – Discrimination Testing of a Rule-Based Software System

The program you are testing is the `loan.py` file which is inside the home directory of twayDiscriminationTesting.zip shown in Figure 1. This is a rule-based decision-making

program that decides if an individual is getting a loan or not. Before completing the following steps, fill in the pre-survey.

```
import sys
gender = sys.argv[1]
race = sys.argv[2]
income = sys.argv[3]

# first case
if gender == "gen1":
        print ("1")
elif race != "green":
        if income == "0...50000":
                print ("0")
        else:
                print ("1")
else:
        if income == "0...50000" or income == "50001...100000":
                print ("0")
        else:
                print ("1")
```

Figure 1: Code snippet of loan program (loan.py)

Steps:

1. Unzip the earlier downloaded `twayDiscriminationTesting.zip` file
2. Go to `data/ACTS_param/loan.txt` in the folder with following command
   - `cd twayDiscriminationTesting`
   - `cd Data`
   - `cd ACTS_param`
3. Define the input parameter for the software under test in the `loan.txt` file. Please ensure the corresponding updates are implemented within the `loan.py` source file accordingly.

   4. Run ACTS to generate test cases in twayDiscriminationTesting directory:
   ACTS generates 2-way combinatorial test cases covering combinations of the input parameters, mainly the protected attributes. Now type the following command to go to the home directory (twayDiscriminationTesting) and generate tests:
   - `cd ..`
   - `cd ..`
   - `java -Dalgo=ipog -Ddoi=2 -Doutput=csv -jar ACTS3/acts_3.0.jar Data/ACTS_param/loan.txt Data/tWay_Abstract_TC/loan_2way.csv`

The tests will be saved in folder:

`Data/tWay_Abstract_TC/loan_2way.csv`

5. Based on generated tests in earlier step, run `run_loan_fairness.py from` the home directory (twayDiscriminationTesting) with following command to observe possible discriminatory instance in the program:

- `Python run_loan_fairness.py`

The script reports differences in favorable outcome rates across demographic groups, which can be examined for potential discriminatory behavior

Deliverables

For full credit, along with the pre-survey:

1. Submit the post survey for part 1.

Part II – Discrimination Testing of a Data-Driven Software System

The program you are testing is the `run_lr_fairness.py` file in the home directory(twayDiscriminationTesting). It uses a logistic regression model on the dataset in `adult_biased_200.csv` file. It predicts whether an individual's annual income exceeds $50K shown in Figure 3.

```
# ── 1. Train logistic regression on biased dataset ──────────────────────
df = pd.read_csv("adult_biased_200.csv")

df_model = df.copy()
le_edu  = LabelEncoder()
le_race = LabelEncoder()
le_sex  = LabelEncoder()

df_model['education-num'] = le_edu.fit_transform(df_model['education-num'])
df_model['race']          = le_race.fit_transform(df_model['race'])
df_model['sex']           = le_sex.fit_transform(df_model['sex'])

features = ['age', 'education-num', 'race', 'sex',
            'capital-gain', 'capital-loss', 'hours-per-week']
X = df_model[features]
y = df_model['pred_label']

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.3, random_state=42)

model = LogisticRegression(max_iter=1000, random_state=42)
model.fit(X_train, y_train)

print("=" * 60)
print("MODEL PERFORMANCE")
print("=" * 60)
print(f"  Accuracy : {accuracy_score(y_test, model.predict(X_test)):.2f}")
print(classification_report(y_test, model.predict(X_test)))
```

Figure 3: Code snippet of adult income prediction

1. Go to data-> ACTS_param -> adult.txt
   - `cd twayDiscriminationTesting`
   - `cd Data`
   - `cd ACTS_param`

2. Define the input parameter for the software under test in `adult.txt` file. For this one, just define the protected attribute values only. Please ensure the corresponding input field values are present within the `adult_biased_200.csv` file.

3. Run the `run_lr_fairness.py` file using command:
   Python `run_lr_fairness.py`

```
# ── 2. Generate ACTS combinatorial test cases ──────────────────────────
print("=" * 60)
print("GENERATING ACTS TEST CASES")
print("=" * 60)

subprocess.call([
    'java', '-Dalgo=ipog', '-Ddoi=2', '-Doutput=csv',
    '-jar', 'ACTS3/acts_3.0.jar',
    'Data/ACTS_param/adult.txt',
    'Data/tWay_Abstract_TC/adult_2way.csv'
])
```

Figure 5: Code snippet of using ACTS for test generation

4. Unlike earlier, this will generate both testcases and fairness reports through the same command in step 3 and shown in Figure5. The tests will be saved in: `Data/tWay_Abstract_TC/ adult_2way.csv`

Deliverables

For full credit, fill up following form:

1. Submit the post survey for part 2.
2. Submit this survey for your overall experience regarding discrimination testing.
3. Please submit your generated test, test result and intended action on discriminatory results in a pdf file.